WildTrace 技术说明书（v2.2）

> 面向赛题《OpenHarmony 开源代码漏洞挖掘》交付件二。本文严格区分**已实现并验证** 的能力与**设计中/计划中**的能力（各节状态标注），不含未经验证的质量主张。 基线：master 分支 `29df68f`（2026-10-22），**1,248 项用例**（`unittest` 发现计数）， CI（Ubuntu）全绿。 v1.0（2026-09-30，基线 `1c80e2d`）收录：自动化分析管线（Autopilot v3）、跨函数 UAF、Joern CPG 桥接、LLM PoC 生成器、版本分支影响矩阵（kb 联接）、交付件组装器。 v2.0（2026-10-17）增补：P6 状态模型与可证明性排序（§3.13）、检测形态扩展 与函数摘要 + 严格修复对回放（§3.14）、LLM PoC 四步闭环含编译反馈环与真库 验证（§3.15）、Joern 端到端实测与双引擎交叉（§3.16）、自足构建链与真库 malformed 验证管线（§3.17）、分支钉版实证与影响矩阵校准（§3.18）、 证据质量矩阵与开放候选台账（§5/附录）。 **v2.1（2026-10-18）增补**：P8 分支/闭包敏感化（§3.19）、种子化召回基准 v2 与 召回守恒账本（§3.20）、LLM 单模型全量审查执行完毕（§3.4/§5）、双引擎函数粒度 并集实测（§3.16）、影响矩阵证据分层（§3.18）、交叉复核 F01-F08 全项响应 （守卫确认收窄、抑制收窄、基准归属修正、验证器状态拆分——§4.1/§7）。 **v2.2（2026-10-22）增补**：LLM 起草的自模仿守卫与 Chat-Completions 通道（§3.9）、 PoC 四道闸门——文本/编译/运行/帧归因与 `wrong_site` 第四档（§3.15）、 双平台守卫堆 harness（§3.15.1）、分支判定按证据强度分层（§3.18）、 模块地图全量化（§2.3 逐模块登记 91 个模块，不再是节选）。 测试基线同步为 1,248 项（原 1,191 为 10-18 数字）。

---

1. 概述

1.1 定位

WildTrace 是一套**本地、离线优先、证据可审计**的 OpenHarmony C/C++ 漏洞分析工具， 覆盖"静态候选 → 结构化研判 → 授权隔离执行 → 运行级证据 → 独立复现 → 报告"全链， 并在此之上提供**端到端自动化分析管线**（Autopilot）与**比赛交付件组装**能力。 所有自动产出物一律为 **candidate/lead（候选）**，漏洞确认只能由人工审核完成—— 这是贯穿全部模块的第一设计约束。

1.2 设计原则

1. **证据优先**：每个结论绑定源码快照摘要、命令、退出码、产物哈希；证据不可变 （只追加存储、独占创建、拒绝符号链接）。
2. **失败即失败**：构建失败、传输失败、输入变化都不会被粉饰为成功结果；离线 核验器拒绝不完整的运行记录。
3. **最小权限**：扫描默认不联网、不执行目标代码、不接触凭据；唯一的执行通道由 双人签署的范围化授权书打开。LLM 通道为显式选配且经注入/外泄加固（§3.4、§3.9）。
4. **有界性**：各模块按实现约束输入与运行资源；快照限制文件数、目录项数和总字节 数；LLM 上下文读取有字节上限。
5. **不发明数字**（v1.0 新增）：影响矩阵只声明有证据的判定格（§3.10）；报告只 投影 case.json 显式字段；确认漏洞数口径固定且不做推断。2026-09-29 审计曾发现 三处"声明与代码不符"（跨组件基线兜底伪造全红矩阵、报告硬编码全分支受影响、 "11 层"实为死代码），已全部修复并由回归测试钉死。

1.3 能力总览

能力	状态	验证证据
多引擎静态分析（7 层，§3.1）	✅ 已实现	44 个真实 OH 源文件分析、35 条诊断入 29 个复核组
函数作用域规则引擎（detect/function_rules）	✅ 已实现	三规则（整数溢出/空指针解引用/固定缓冲拷贝）+ 守卫精度；34 项配对真假基准；削减正则粗筛两类已知误报，入 benchmark 对比
自动化管线 Autopilot v3（§3.8）	✅ 已实现	dsoftbus 742K 行实测（历史口径，见 §5）；1016 项回归含专项测试
跨函数 UAF/双重释放（§3.8.2）	✅ 已实现	位置别名 + 局部变量追踪 + (callee,var) 去重；专项测试 20+ 项
Joern CPG 深度分析（§3.8.3）	✅ 已实现（--joern 选配）	WSL /opt/joern 实测 dsoftbus；异常安全降级
LLM PoC 生成器（§3.9）	✅ 已实现并加固	路径包含/注入防御/输出屏蔽 16 项安全测试
版本分支影响矩阵 + kb 联接（§3.10）	✅ 已实现	20 案 × 6 分支 = 105 证实 / 15 证实不受影响 / 0 待定（全部可复算）
CVE 知识库三层 + 版本探测（§3.6）	✅ 已实现	全量层 375,265 条；靶区子库 446 条（kb 联接数据源）
隔离执行与运行级证据（§3.5）	✅ 已实现并实战	cJSON 案例矩阵 26/26；第二人独立复现通过
运行矩阵离线复算（verify-runtime）	✅ 已实现	复算=零执行
有界 LLM 审查（§3.4）	✅ 全量执行完毕（单模型）	本机重建 26 组队列，glm-5.3 逐组有效草稿 26/26，final_verdict 全 undetermined；双模型交叉因仅存 GLM 密钥不可行（如实）
UAF 生命周期规则（§3.3）	✅ 已实现	已知 cJSON 案例回验 + 反例语料
候选人工复核队列（triage）	已实现五类队列	保留每条告警，不自动证明可达性与影响
交付件组装（vuln_report / competition_report，§3.11）	✅ 已实现	20 案报告自动生成；证据引用 16/20 案
工具链预检（toolchain preflight）	✅ 已实现	`python -m ohvul.toolchain --output ...`
执行沙箱自动化（§3.5 扩展）	📐 设计稿已合入	P0–P2 待实施
单模型全量审查（glm-5.3）	📐 队列与 plan 就绪	26 组本机重建（2026-10-17）；双模型交叉因仅 GLM 密钥不可行，如实降级为单模型
OH 真实库构建与平台验证	IPC 库级证实；产品验证待完成	`REAL_BUILD_PROBE_20260925.md`；库级≠平台复现
P6 状态模型与可证明性排序（§3.13）	✅ 已实现	Null/NonNull/MaybeNull 时间线；dsoftbus 漏斗 2,662→13（语义分层）；51 项契约测试
检测形态扩展 + 函数摘要（§3.14）	✅ 已实现	GT1 局部/GT2 成员/GT3 宏展开指针；三个严格修复对回放命中→消失零误伤
LLM PoC 四道闸门（§3.9/§3.15）	✅ 已实现	文本形态分类 + 链接集自推断编译 + 运行三态（静默腐蚀保护）+ 帧归因 `wrong_site`；端到端 ~30s/案

能力	状态	验证证据
Joern 端到端 + 双引擎交叉（§3.16）	✅ 已实测	available=true、findings=1（J6_null_deref）；P6+J6 双层
自足构建链 + 真库 malformed 管线（§3.17）	✅ 已实现	oh_build 工具包四步管线；udmf/bt#19 真库 SIGSEGV 实证（exit 139 + 帧归因）
分支钉版实证（§3.18）	✅ 已实现	manifest 钉版 + gitcode README 实证；影响矩阵 20 案六分支全判定（105 格），待定格清零
P8 分支/闭包敏感化（§3.19）	✅ 已实现	dsoftbus 448→69（移除 379 全为裁定形态）；交叉复核 F04 三处收窄后反例回归锁定
种子化召回基准 v2（§3.20）	✅ 已实现	11 正向种子（独立文件归属）+ 5 负向 silent_for：召回 11/11、violations=0
召回守恒账本（§3.20）	✅ 工具就绪	分目录漏斗 + 抑制原因分布；口径声明（单位不同不做机器守恒断言）

2. 总体架构

2.1 工作流状态机

`ohvul/workflow/fsm.py`：状态集 `intake → static_scan → local_verification → triage → report → human_review → completed`（另有 blocked / 重试预算 ≤3）。关键约束：

- 迁移合法性由 TRANSITIONS 表硬编码；`local_verification → triage` 必须携带 checkpoint/result/harness 三元组并通过 `load_verified_checkpoint` 校验；
- blocked 恢复只能回到 intake/static_scan（有界恢复）；重试预算在迁移内强制；
- 检查点含 `source_snapshot_sha256`：扫描前后比对，源码期间变化即拒绝产出。

2.2 证据链模型

一切产物走同一条证据管道：

- **EvidenceBundle**（`domain/evidence.py`）：内容寻址（`EB-<sha256>`），源位置、环境指纹（execution/network 标志）、artifacts 摘要列表；
- **联合校验**（`verify.py` / `verify_joint.py`）：finding↔bundle 摘要逐字段一致；报告投影字段与扫描结果全等（`report_verify`）；
- **归档封存**：日志、检查点、报告归档后经 `journal_archive_verify` / `archive_seal_record` 生成不可覆盖的关联标记；
- **verification_link**：task_id + checkpoint 摘要 + harness 记录三元绑定。

2.3 模块地图（代码功能实现）

`ohvul` 包共 **91 个模块**（不含 6 个 `__init__`），按调用方向分八层。下表**逐模块登记**（不是节选），每行给出该文件在流程中的位置与它不该被当成什么——后者是防止误读的关键。

2.3.1 入口与编排层（10）

模块	职责	对应
`cli.py`	离线扫描入口：扫描 → 检查点/日志/报告；产物必须在源码树之外（越界即报错退出）	§2.1

模块	职责	对应
`autopilot.py`	**端到端管线 v3**：10 层规则聚合、0–100 置信度与因子分解、PoC 模板、统一报告	§3.8
`component_batch.py`	逐组件串行批跑（CVE 富集 6 组件即此路径），无断点续跑	§3.8
`toolchain.py`	工具链预检：WSL/Clang/clang-tidy 真实探针，required/optional 分级（演示第一步）	§4.3
`recover.py`	从检查点恢复**工作流**，不重跑审计、不重算结论	§2.1
`shards.py`	生成确定性源码分片清单；只读目录结构，不读文件内容	§3.1
`engine_cli.py`	单次显式 WSL Clang 分析，输出 JSON	§3.1
`engine_shards.py`	按分片清单批量驱动本地 WSL Clang 适配器	§3.1
`workflow/fsm.py`	工作流状态机：7 状态 + 迁移表 + 重试预算（§2.1）	§2.1
`workflow/journal.py`	追加式审计日志；**日志提交标记本身不是证明**	§2.2

2.3.2 检测规则层（13）

模块	职责	对应
`detect/scanner.py`	内置规则扫描器：CWE 规则表（CWE-190/476/122…）+ 证据 bundle 生成	§3.8
`detect/lexical.py`	词法复核过滤器；**不是 C/C++ 解析器**，不做安全证明	§3.12
`detect/function_rules.py`	函数作用域规则草稿；产出候选，永不产出漏洞结论	§3.8
`detect/components.py`	组件识别（路径 → 组件名 → 元数据）	§3.8
`detect/vuln_metadata.py`	CVE 元数据保守匹配；**版本命中不等于影响确认**	§3.6
`semantic_facts.py`	C/C++ 语义事实抽取（长度来源、容量关系、所有权）	§3.2
`semantic_rules.py`	长度传播候选规则；无 CFG/数据流证明，只作保守筛选	§3.2
`semantic_syntax.py`	有界词法输入、括号配平；不构建 AST	§3.12
`source_structure.py`	有界词法区间与函数边界；**不是 AST，也不是调用图**	§3.12
`source_scope.py`	路径契约与只读路径工具；**不是操作系统沙箱**	§3.5
`source_snapshot.py`	源码快照 SHA-256（扫描前后比对，期间变动即拒绝产出）	§2.1
`cross_uaf.py`	跨函数 UAF / double-free / 位置别名追踪（P8-Cross），按 (callee,变量) 去重	§3.3
`ffi_boundary_audit.py`	FFI 裸数组边界初筛（成员名与裸数组同名即导航候选）	§3.8

2.3.3 编译器事实层（10）

模块	职责	对应
`clang_stream.py`	流式解析 Clang JSON，只保留主文件函数体与函数声明	§3.2
`clang_facts.py`	编译器派生事实与窄口径对象大小检查	§3.2
`clang_flow.py`	有界整数传递与分支约束	§3.2
`clang_ranges.py`	选定表达式的有限整数区间	§3.2
`compiler_analysis.py`	固定版本、受校验组件输入的显式 AST 分析	§3.2
`compile_database.py`	把显式编译参数向量导入 profile 草稿	§3.1

模块	职责	对应
`engine_adapter.py`	分析引擎适配契约与本地工具发现	§3.1
`engine_profile.py`	组件分析 profile 校验（来源、提交、文件清单、include、参数）	§3.1
`engine_benchmark.py`	人工标注下的引擎候选指标（仅离线）	§3.1
`joern_bridge.py`	Joern CPG 桥接（`--joern` 第 11 层，不可用时降级不中断）	§3.16

2.3.4 验证与证据层（12）

模块	职责	对应
`domain/evidence.py`	EvidenceBundle 内容寻址结构（`EB-<sha256>`）；哈希只证明未变更，不证明真实性	§2.2
`local_harness.py`	合成输入本地 harness；**不执行被审目标代码**	§3.5
`local_recheck.py`	离线源码证据复核，不执行目标	§3.5
`runtime_evidence.py`	把已有候选报告与外部收集的运行证据桥接	§3.5
`isolation_contract.py`	离线合成 harness 的 I/O 契约校验	§3.5
`harness_record.py`	harness 记录 schema 校验	§3.5
`verification_gate.py`	「是否允许发起本地安全验证」的确定性闸门	§3.5
`verification_link.py`	task_id + checkpoint 摘要 + harness 记录三元绑定	§2.2
`verify.py`	已存审计结果与引用的离线校验	§2.2
`verify_joint.py`	checkpoint / result / harness 三方联合校验 CLI	§2.2
`benchmark.py`	配对真假的候选规则基准（GT1/GT2/GT3 回放）	§3.14
`performance.py`	合成输入的性能基准（不等于产品扫描吞吐）	§5

2.3.5 LLM 与 PoC 层（7）

模块	职责	对应
`llm_adapter.py`	有界远程起草：端点/模型闭集白名单、禁重定向、字节上限、凭证筛查	§3.9.1
`llm_evidence.py`	把已验证的本地片段确定性投影进模型输入（有界）	§3.4
`llm_review.py`	有界 LLM 审查；输出恒为 proposal，`final_verdict=undetermined`	§3.4
`llm_verify.py`	编译-执行反馈环（四道闸门的前身入口，保留兼容）	§3.15
`llm_poc.py`	PoC 生成 + 自模仿守卫（头文件豁免、`_defines_function`）+ 拒收回灌	§3.9
`poc_verify.py`	闸门一至三：形态分类、链接集自推断编译、运行三态（静默腐蚀保护）	§3.15
`poc_real.py`	闸门四：真库帧归因（WSL + musl + gdb + `addr2line`），支撑 `wrong_site`	§3.15

2.3.6 审查与校验层（20）

模块	职责	对应
`candidate_triage.py`	离线待办队列分桶（人工复核/需上下文/语法线索/质量/测试支援）	§3.8
`review_queue.py`	绑定已存批次证据的确定性离线审查队列	§3.4
`review_context.py`	为已校验队列构建有界源码窗口	§3.4

模块	职责	对应
`review_evidence.py`	诊断文件内的函数/宏上下文扩展	§3.4
`review_handoff.py`	证据绑定的后续任务单；不执行模型建议	§3.4
`review_record.py`	独立于结论的人工复核记录	§3.4
`review_verify.py`	复核记录对照候选报告的只读校验	§3.4
`review_batch_verify.py`	复核批次对照候选报告的只读校验	§3.4
`review_chain_verify.py`	报告—批次—日志三者链路校验	§2.2
`report_verify.py`	报告投影字段与扫描结果全等的只读校验	§2.2
`journal_archive.py`	导出任务审计历史（不改原日志）	§2.2
`journal_archive_verify.py`	归档日志的只读校验	§2.2
`journal_status.py`	日志恢复状态的只读检查	§2.2
`archive_seal_record.py`	生成独立封存标记（无签名、不改权限）	§2.2
`archive_seal_verify.py`	封存标记只读校验	§2.2
`profile_capture.py`	从显式本地输入捕获仓库钉版	§3.18
`knowledge_base.py`	版本化本地知识库校验（无网络）	§3.6
`kb_sync.py`	CVE 知识库同步；构建阶段不扫目标	§3.6
`branch_impact.py`	两次已存扫描结果对比（不碰 Git、不执行目标）	§3.10
`git_scope.py`	固定本地 Git 分析范围校验（无网络访问）	§3.18

2.3.7 交付件层（8）

模块	职责	对应
`impact_matrix.py`	版本分支影响矩阵（kb 联接 + 钉版 + 符号探测），按证据强度分列	§3.10
`impact_report.py`	分支影响报告的只读投影（Phase 5）	§3.10
`vuln_report.py`	分案完整报告生成器（五要素 + PoC + 补丁）（交付件 4）	§3.11
`competition_report.py`	异构 case.json 的防御性投影 + 证据完整度六维矩阵	§3.11
`report/schema.py`	报告 schema 定义与校验	§3.11
`report/review.py`	人工复核记录的报表投影	§3.11
`report/review_batch.py`	复核批次的报表投影	§3.11
`report/archive_seal.py`	报告归档封存的报表侧实现	§2.2

2.3.8 工作台层（11）

模块	职责	对应
`workbench/launch.py`	本地工作台启动（默认 `127.0.0.1:8765`）	`USAGE.md` §6
`workbench/app.py`	HTTP 路由与页面装配	`USAGE.md` §6
`workbench/data.py`	案例与扫描数据的装载层	`USAGE.md` §6

模块	职责	对应
`workbench/store.py`	本地数据存储（默认 `.cache/workbench`）	`USAGE.md` §6
`workbench/jobs.py`	后台作业登记与状态	`USAGE.md` §6
`workbench/worker.py`	作业执行器（长任务不阻塞请求）	`USAGE.md` §6
`workbench/triage.py`	扫描结果的确定性复核队列	`USAGE.md` §6
`workbench/casebook.py`	案例叠加层（版本化；仓库内案例记录保持不可变）	`USAGE.md` §6
`workbench/projects.py`	项目路径与只读探针（WSL 路径在记录中保持原生）	`USAGE.md` §6
`workbench/project_io.py`	在选定项目环境内执行的只读操作	`USAGE.md` §6
`workbench/project_scan.py`	把只读源码范围绑定到隔离的扫描输入副本	`USAGE.md` §6

2.3.9 非 Python 交付物

路径	内容
`tools/harness/`	守卫堆（Windows 预处理器改名 / Linux `LD_PRELOAD`）与 OHOS 真库 runner（§3.15.1）
`tools/perf_measure.py`	性能测量与复跑（§9 复跑命令）
`cases/`	20 案 case.json、PoC、patch.diff、证据（66 个文件入库）
`patches/`	20 份补丁镜像 + 索引（与 `cases/*/patch.diff` 逐字节一致）
`kb/`	靶区 CVE 记录（records.json）与分支钉版（branch_pins.json）
`bench/`	反例语料与配对基准
`frontend/`	工作台前端（Node 构建）

---

3. 关键技术原理

3.1 多引擎静态分析栈（已实现）

七层互补，全部产出统一 schema 0.3 候选（带 EvidenceBundle）：

1. **内置规则**（正则初筛，定位为粗筛不作为依据）；
2. **语义层**：有界词法解析（结构剥离：注释/字符串/预处理行；括号平衡参数提取）+ 跨函数长度传播候选；
3. **Clang 静态分析器**（WSL 隔离运行，plist 归一化）；
4. **clang-tidy**（文本诊断归一化，检查器集合显式声明）；
5. **Semgrep 污点**（本地规则文件强制、显式文件清单）；
6. **编译器 AST 事实层**：固定版本 clang `--analyze` 流式 AST 解析，进程隔离；
7. **标量流 + 区间域**：AST 上的保守数据流（§3.2）。

统一约束：`_normalize_engine_path` 将 /mnt 形态路径统一为相对根并拒绝越界；引擎结果经 `engine_result_to_audit_result` 投影进主 schema 并可合并去重。

3.2 编译器背书的事实层与区间抽象域（已实现）

- **进程隔离**（`compiler_analysis._command`）：固定编译器路径 + 外层 `timeout --kill-after` + `env -u` 清除 CPATH 等 5 个环境注入向量 + `--no-default-config` 禁配置文件发现；shell=False、stdin 封死；双流有界读取 线程（超限即 kill，字节计数与 sha256 入证据）。
- **精确值标量流**（`clang_flow`）：仅自动存储期整型局部变量；**kill-on-unknown** ——任何调用、未知存储、取地址、非结构化控制即清空环境；循环入口杀死循环前 赋值；预算墙（10 万节点/128 深/4096 绑定/64 证据源）。
- **区间抽象域**（`clang_ranges`）：类型域约束下的区间算术（四角乘积、比较三值 逻辑）；转换语义是正确性核心——有符号越界返回 None、无符号仅同模数块内回绕； 混合符号比较反推与非保值转换约束全部有测试钉死。
- **类型尺寸**（`clang_facts.size_of`）：括号指针声明符一律指针尺寸；不可建模 形态返回未知——解析失败绝不产生看似精确的错误数。

3.3 UAF 生命周期规则与已知案例回验（已实现）

两条窄规则（#34）：同句柄 use-after-delete 与同 pointee 类型别名使用。 **方法论**：每条新规则先在 `bench/lifecycle_counterexamples/` 配对语料（含真实 cJSON 正例与安全反例）上回验再上线。

3.4 有界 LLM 审查（单模型全量执行完毕，26/26 草稿全 undetermined）

- **传输**（`llm_adapter`）：HTTPS-only 白名单端点、模型白名单、密钥仅运行时 读取、请求与消息双向凭证形态拒绝、无流/无重试/无重定向、尺寸与超时上限； 畸形 provider 响应一律收敛为 `LlmTransportError`（不裸逃异常）。
- **审查契约**（`llm_review`）：五维评估问题；`independent_validation` 强制 unknown；系统提示内建注入防御；`final_verdict` 恒为 undetermined。
- **交接**（`review_handoff`）：LLM 草稿投影为"带证据引用的开放任务"，全部标记 untrusted_model_proposal。
- **单模型全量审查**（glm-5.3）：26 组队列本机重建（`LLM_REVIEW_RERUN_20261017.md`） 并**执行完毕**（`LLM_REVIEW_RUN_20261017.md`）——46 次调用 / 66,782 tokens （provider 上报口径），分轮补跑克服通道抖动，终轮 7/7 全绿且离线 verify completed；26/26 草稿 `final_verdict` 全 undetermined、确认漏洞数 0；建议分布 24 lower_priority / 1 manual_review / 1 needs_context；五问矩阵中 input_control 24 unknown 与工具自身调用方分析缺口同构（如实保留）。双模型交叉 原设计为 glm-5.3 × gpt-6-astra，现仅存 GLM 密钥，如实降级为单模型。

3.5 隔离执行与运行级证据（已实现并实战验证）

以双人签署的《隔离执行环境授权书》为唯一通道：

- **隔离**：bubblewrap `--unshare-all --cap-drop ALL`，只读绑定源码与工具链， tmpfs /tmp、clearenv、loopback-only；运行前隔离探针验证；prlimit 限制。
- **判定**：崩溃分类完全机器判定（ASan 报告 + 访问轨迹 + 日志特征）； 功能对照要求零 sanitizer 报错 + CASE_OK 标记。
- **离线复算**（verify-runtime）：不执行任何代码复算每行分类。
- **实战验证**：cJSON 公开案例（CVE-2026-87933 库级）矩阵 26/26，含第二复现人 独立环境复现（26/26，结构全等）。边界声明：已知公开案例的库级验证，非新发现、 非产品级结论。

3.6 CVE 知识库三层与版本探测（已实现）

层	内容	规模与状态
全量层	cvelistV5 归一化	375,265 条 / 188 分片，本地构建+verify（体积原因不入库）
精选层	OH 官方公告 CVE，双人复核	72 条入库，含逐分支修复 commit + diff 边车
靶区子库	19 个高价值 third_party 组件	446 条入库（openssl 133 / curl 98 / ffmpeg 80…）

靶区子库自 2026-09-29 起兼任影响矩阵的判定数据源（§3.10）。版本探测 （`tools/kb_version_probe.py`）与 OSS-Fuzz OSV 层的边界同 v0.9 记录。

3.7 安全边界体系

默认禁令（不执行目标代码/不联网/不自动提交）→ 唯一例外 = 双签授权书； LLM 凭证永不入库入日志；CI 含密钥形态扫描。LLM PoC 生成器在选配时叠加： 路径包含检查、提示注入防御（围栏消解 + 摘要绑定 + 不可信标注）、输出危险 API 屏蔽（§3.9）。

3.8 自动化分析管线 Autopilot v3（已实现）

```
python -m ohvul.autopilot <source_root> --output <dir> [--min-confidence N] [--joern]
```

一条命令完成：文件面发现（CJ FFI / 通用 C/C++ 分类）→ 多层扫描 → 置信度排序 → PoC 模板生成 → 统一 JSON+Markdown 报告。

3.8.1 层级结构（默认 10 层，--joern 时 11 层）

1. P1–P3：CJ FFI 面三族（空指针+长度 unchecked、整型截断、多级解引用）；
2. P4–P9：通用 C/C++ 六族（无界拷贝、栈溢出、空指针、分配溢出、UAF、格式串）；
3. 第 10 层：跨函数 UAF/双重释放（§3.8.2）；
4. 第 11 层（选配 `--joern`）：Joern CPG 深度分析（§3.8.3）。

`summary.scanners` 如实列出执行过的层； `conf>=70` 为精确计数。

3.8.2 跨函数 UAF（cross_uaf）

三种单函数 P8 看不到的模式：A（caller 释放后传给 callee 解引用）、B（callee 释放后 caller 继续使用）、C（双重释放）。要点：

- **位置别名解析**：caller 的 `ptr` 映射 callee 的第 N 个参数（参数名可不同）； 函数指针/数组/未命名参数保留位置空槽，后续参数不错位；
- **局部变量追踪**（`freed_any`）： `malloc → free → 传参` 的局部指针 UAF 可检出 （callee 侧判定仍严格限参数，避免同名局部误报）；
- **去重语义**：每个模式按 (callee, 变量) 只发一条——每个调用点一条的旧实现 在 3000 次 `sink(a)` 调用上产出 3000 条重复发现；
- **宏免疫**：`#define` 宏体不解析为函数，dsoftbus `LIST_FOR_EACH_ENTRY_SAFE` 类宏误报已实证为零并有回归测试钉死；
- **同名多定义**：事实完全相同的同名函数保留（归属无关），不同则判歧义丢弃 （不静默 last-writer-wins）。

3.8.3 Joern CPG 桥接（选配）

`--joern` 显式启用：J4（无界拷贝）/J6（空指针路径敏感）/J8（UAF 跨定义）三个 CPG 查询，发现转换为管线 Finding（CWE/严重度/置信度保留），报告含 joern 统计块。 默认运行不触碰 WSL；不可用时警告降级不崩溃。工程细节：

- 全部 subprocess 异常安全（WSL 缺失返回 False 而非抛异常）、UTF-8 容错解码；
- Scala 侧 JSON 转义（含引号/反斜杠的代码不再静默丢发现）、路径 shlex 引用；
- CPG 二进制写入分析环境 `/tmp/wildtrace-cpg/<源根哈希>.bin`（不污染被审计树， 按源根命名跨运行复用； `remove_cpg()` 回收）；
- CPG 构建失败只试一次（旧版每查询重试最长 33 分钟）。

3.8.4 置信度打分

每条发现 0–100 分并附人类可读因子分解（Coverity 式统计优先级）。打分测试 正确的变量（如 P4 的 size 变量、P9 的格式参数位）；调用方空检查降分保留而非 丢弃；跨函数发现携带别名证据加分。`lead ≠ 漏洞`：置信度是排序优先级，不是 准确率保证。

3.9 LLM PoC 生成器（已实现并加固，选配）

```
# 默认通道：GLM（Anthropic Messages）
python -m ohvul.llm_poc findings.json <source_root> --output llm_pocs

# Chat-Completions 通道：DeepSeek
python -m ohvul.llm_poc findings.json <source_root> --output llm_pocs \
    --base-url https://api.deepseek.com/v1 --model deepseek-chat
```

把 autopilot lead + 源码上下文交给 LLM 生成可编译 C PoC（从"带 TODO 的模板" 升级为"真实代码"）。安全契约（2026-09-29 审计后加固，16 项专项测试）：

- **路径包含**：`finding['file']` 必须解析在 source_root 内（`../` 逃逸与绝对 路径覆盖一律拒绝）——旧实现可把任意本地文件前 3000 字符发往远端端点；
- **注入防御**：扫描源码与 findings 字段按不可信数据处理（围栏序列消解 + sha256 摘要绑定 + "UNTRUSTED DATA" 标注 + 系统提示证据契约）；
- **输出屏蔽**：危险 API 黑名单（system/popen/exec*/fork/unlink/remove、 /bin/sh、curl/wget/nc）命中即不落盘；截断响应（finish_reason=length）拒绝；
- **批量鲁棒**：逐项异常隔离、摘要 finally 写出、429 退避、文件名哈希防碰撞；
- **有界读取**：上下文读取 256 KiB 封顶。

3.9.1 双通道传输与端点白名单

所有 LLM 入口（PoC 起草、审查、分析）统一走 `ohvul/llm_adapter.py`，不直连 HTTP：

协议	白名单端点	白名单模型
Anthropic Messages（默认）	`api.z.ai/api/anthropic`、`open.bigmodel.cn/api/anthropic`	`glm-5.3`
OpenAI Chat-Completions	`api.deepseek.com/v1`	`deepseek-chat`、`deepseek-reasoner`

端点与模型都是**闭集**：`validate_settings()` 对不在集合内的 base_url 或模型直接 抛错，不做"拼一个 URL 就发"的兜底。传输层约束：

- **禁重定向**：注册 `_NoRedirect` handler，任何 3xx 都转为 `redirect_rejected`， 避免 Authorization 头被带到白名单以外的站点；
- **不重试**：单次有界请求，失败即返回错误码，由上层决定是否重写；
- **字节上限**：请求 64 KiB / 响应 256 KiB / 单条内容 32 KiB / 超时 300 s；
- **凭证筛查**：`reject_credentials()` 在出入两个方向扫 `sk-` 形态、 PEM 私钥头、`Authorization: Bearer`，命中即拒收，错误码不含 URL 与凭证；
- **逻辑载荷绑定**：`canonical_request_payload()` 固定 `stream=False` 、 `temperature=0` 、 `response_format=json_object`，摘要只绑模型/消息/额度， 不绑线路封装，便于事后复算。

3.9.2 自模仿守卫（2026-10-21，根因修复）

首轮实测（DeepSeek + Chat-Completions，cJSON 目标）**10/10 草稿全是"自实现副本"**： PoC 不调用库，而是把 `static` 目标函数体抄进 `main()` 再喂坏输入——这种崩在副本里 的崩溃是套套逻辑，对被测库零证据。根因不在模型而在校验规则：旧规则要求 "目标函数名必须出现在 PoC 里"，而目标是文件内 `static` 函数，**外部不可调用**， 于是模型唯一能满足规则的写法就是抄函数体。

修复是两件事，缺一不可：

1. **头文件豁免**（`_project_headers()`）：解析目标文件的 `#include`，取项目自有 头文件基名（排除 `stdio.h`/`stdlib.h` 等标准头）。草稿若 include 了项目自己的 头（如 `cJSON.h`），允许它经公共 API 间接触达，不再强制内部函数名字面出现；
2. **定义检测而非出现检测**（`_defines_function()`）：判断草稿是否**定义**了目标 函数。不是单条正则——`if (print(x)) {` 这类嵌套调用会被朴素模式误判成定义， 因此按声明形态逐项排除：前缀含 `=`（赋值）、前缀含括号且不是导出宏 （`CJSON_PUBLIC(cJSON *)` 这类合法导出宏单独放行）、首个词是控制字（`if/while/for/switch/return/...`）、括号配平后紧跟 `{` 才算定义。 同一函数被 `poc_verify` 复用（`classify_poc` 直接 import 它），**起草闸门与 验证闸门对"什么叫副本"的定义不可能分歧**。

修复后同一 prompt 的 5/5 草稿走真实 API（`real_link` 形态），不再出现副本型。

3.9.3 拒收回灌与重写预算

拒收原因分两类，只有"模型改一版就能修好"的才重写：

- **可重写**（`_RETRYABLE_REJECTIONS`）： `reimplements_target_function` 、 `target_function_*` 、 `no_main_function`、`no_includes`、`dangerous_api`；
- **不重写**：传输失败、响应截断、凭证命中——再问一次只是白烧 token。

重写预算默认 `retries=1`（`retries=0` 退化为单发模式），失败原因原文进下一轮 提示词。第四、五道闸门（编译反馈、运行反馈）通过 `compile_hook` / `runtime_hook` 注入，见 §3.15。

3.10 版本分支影响矩阵与 kb 联接（已实现）

```
python -m ohvul.impact_matrix --cases cases --output docs/deliverables/IMPACT_MATRIX
```

20 案 × 6 分支（v4.1–master）。判定规则**证据分级、绝不推断**：

1. **kb 联接（首选）**：按 CVE 联接 `kb/third_party_targets/records.json` （cvelistV5）的上游受影响/修复版本，按 cvelist 边界语义逐分支判定；
2. case.json 结构化受影响范围（CVE 5.0 语义）；
3. 版本等价传播（分支基线与实测验证基线同版本）；

4. OH 原生代码仅计源级验证过的分支；**绝不跨组件借用基线**。

当前交付件结果：🔴 证实受影响 105 格 / — 证实不受影响 15 格 / ？待定 0 格（105 格按证据强度：检出级 20 / 钉版联接 72 / 符号探测 13；矩阵表分三列计数）（20 案 × 6 分支 = 120 格，与 `IMPACT_MATRIX.json` 逐格可复算）。v2.1 前为 18 案、88 / 6 / 14 口径；2026-10 补全 cJSON 六分支基线并完成原生四案符号探测后，20 案全部落定，待定格清零。证据强度分层见 §3.18 与矩阵图例脚注：钉版联接类与 符号探测类格子弱于检出/复现级实证，不以同色呈现。 值得一提的负向结论：quirc（cvelist 引入版本 7.0）在 n4.1 / n5.1.6 / n6.1.1 四个基线上**证实不受影响**——旧手写矩阵的全红口径被库内数据证伪；udmf 在 v5.1 分支因 `FfiUDMFSetProperties` 未引入而判 not_affected。

3.11 交付件组装（已实现）

- `ohvul.vuln_report`：分案完整报告（位置与内容、描述、PoC、影响分析、修复 建议）。源码片段承载内容：案例目录 `source_snippet.txt/.c` 嵌入，否则引用已 记录的源码级证据（16/20 案）；已存在输出拒绝静默覆盖（--force）。
- `ohvul.competition_report`：异构 case.json 的防御性投影 + 证据完整度六维矩阵； 空输入大声失败；markdown 先于 JSON 发布；confirmed 数字只投影不推断。

3.12 函数切分的注释安全性（已实现，贯穿 autopilot/cross_uaf）

函数头识别与括号匹配在**长度保真的注释/字符串掩码**上进行：注释中的撇号不再 吞掉整个函数（742K 行级的大面积漏报根源）、注释内 `}` 不再截断函数体、`// {` 不再吞并下一函数、`else if` 与 `#define` 宏不再被当函数；块注释按换行数替换， 报告行号不漂移。

3.13 P6 状态模型与可证明性排序（已实现，导师检测提升路线第 4 步）

对每条 `sptr<>`/指针变量建立 **Null / NonNull / MaybeNull** 三态时间线 （赋值点、判空点、解引用点事件序列），据状态机输出四类规则结论 （确定性问题 / 语义类 / 已知可空路径 / 参数路径）。配套：

- **语义分层漏斗**：dsoftbus 治理后的 2,662 条候选中 P6 族 2,224 条，按语义分层 （Null 确定性 > 语义类 > 已知可空 > 参数 > 基线）收敛至 13 条高价值线索；
- **排序 v2**：语义证据 +10 优先于路径词 +5；每个不发射的访问在抑制日志中 记录原因（可审计、可回归）；
- 51 项 P6 契约测试钉死判定口径。

3.14 检测形态扩展与函数摘要（已实现，严格修复对回放验证）

历史已知漏洞回放暴露三类检测盲点形态，全部实现并回放验证：

形态	修复对提交	回放结果
GT1 局部智能指针（`sptr<> proxy = GetRemoteProxy()` 后裸解引用）	88b2edee	命中→消失，零误伤
GT2 成员指针（`pimpl->`）	100402f0	命中→消失，零误伤
GT3 宏展开取指针	b9fd4075	命中→消失，零误伤

函数摘要（跨函数空指针传播）同步入册；29 个未修复站点经调用方可达性 穷尽分析后裁定（`OPEN_CANDIDATES_LEDGER.md`）。

3.15 PoC 四道闸门：文本 / 编译 / 运行 / 帧归因（已实现）

起草只是第一步。一条草稿要成为证据，必须依次穿过四道闸门；每道闸门都能把它挡回 `undetermined`，而不是"跑起来就算数"。前三道在 `ohvul/poc_verify.py`，第四道 在 `ohvul/poc_real.py`。

```
python -m ohvul.poc_verify llm_pocs --source <source_root> \
    --findings findings.json --output verify_out \
    [--guarded-heap tools/harness] [--frames frames.json]
```

闸门一 · 文本（形态分类）。`classify_poc()` 把草稿分成三形态：

形态	判据	崩溃的含义
`copied_target`	`_defines_function()` 判定草稿自己定义了目标函数	**套套逻辑**——崩在副本里，对被测库零证据
`self_contained`	无项目 include，自足编译	崩在 PoC 自身逻辑，说明不了库的问题
`real_link`	include 了项目头文件	只有这一类的崩溃才可能构成库的证据

闸门二 · 编译（链接集自推断）。草稿不会写"要链 cJSON.c"，findings 也不会。`resolve_link_sources()` 顺着链接器自己的报错走：编译 → 解析 `undefined symbol` / `undefined reference to` / `unresolved external symbol` → 把同目录优先（最多扫 400 个 `.c/.cc/.cpp`）里**定义**这些符号的源文件加进链接集 → 重试。上限 3 轮、 每轮最多新增 4 个文件，不会把整棵树拖进来。编译器发现顺序 `zig cc`（pip wheel， 无需管理员权限，规避 Defender 对 MinGW 的隔离） → `WILDTRACE_CC` → `gcc/clang/cc`； `zig` 首次构建 mingw CRT 会有 `SubCompilationFailed` 偶发，按瞬态错误原命令重试， 不消耗模型重写预算。

闸门三 · 运行（三态判定 + 静默腐蚀保护）。`run_poc()` 限时 10 s，崩溃码取 Windows `0xC0000005` 与 POSIX `139/134/136`。关键设计是**"没崩"不等于"证伪"**：

- P6 空解引用、P9 格式化字符串不崩就是真的没触发；
- P8 UAF/跨函数 UAF/double free、P4 无界拷贝、P5 栈溢出、P7 分配溢出属于 **静默腐蚀**——没有内存检查器时"返回 0"与"发生了 UAF 但读到可读垃圾"外观一致；
- 因此 `memory_checker_available()` 先探 `-fsanitize=address` 能否链接（MinGW 缺 ASan 运行时，会报 `undefined symbol: __asan_report_load8`）；探不到且模式属静默腐蚀类，一律 `undetermined / no_crash_without_memory_checker`，**禁止升格为 refuted**。

闸门四 · 帧归因（`wrong_site` 第四档）。`real_link` + 崩溃 + "在库内"仍是必要 不充分条件：崩在库里不等于崩在被告知的那一行。`poc_real.resolve_frame()` 在 WSL 里用 OHOS musl sysroot（`$BUILD_OUTPUT/obj/third_party/musl`）构建客户端，`gdb -batch` 跑两轮，取崩溃偏移交给 `llvm-addr2line -i -f -C`（对 unstripped 库）符号化，返回逐帧 `{function,file,line}`。比对是**函数级**（`_frame_matches()`）：偏移相等在跨 构建时过严（同一行会漂移），文件+行号在 inline 帧上又会漏；取"稳定下来的外层符号" 做包含比较。

四档判定（`verdict_for()`）：

判定	触发条件	理由码
`confirmed`	real_link + 崩溃 + 帧与 finding 函数吻合	`crash_inside_library`
`wrong_site`	real_link + 崩溃 + 帧落在无关函数	`crash_at_unrelated_frame`
`refuted`	real_link + 跑完 + 库未崩（且有内存检查器覆盖该模式）	`real_link_no_crash`

判定	触发条件	理由码
`undetermined`	未编译/未运行/副本崩溃/静默腐蚀且无检查器/未链库	`build_failed`、`did_not_run`、`copied_target_crash_is_tautology`、`no_crash_without_memory_checker`、`not_linked_against_library`

实测两例说明第四档不是纸面设计：

- **CVE-2026-87933（cJSON）**：同一份 PoC 在默认堆上 `return 0`，在隔离堆下 SIGSEGV——闸门三把它从"看起来证伪"改判 `confirmed`；
- **UDMF**：第一版猜错 ABI，`exit 139` 且崩在库内，但符号化帧是 `SetDataParameters`（`0x16e5d`），而 finding 指的是其下一行的 tag 转换 → `wrong_site`；换到 `0x1718a` 后帧吻合 → `confirmed`。 若只有前三道闸门，"139 + 在库内"会把它误判成 confirmed。

3.15.1 守卫堆：没有 ASan 也要让 UAF 现形

目标平台两个硬约束：MinGW 没有 ASan 运行时，`lld` 又不支持 `--wrap`。剩下唯一 可移植的钩子是预处理器——把标准分配器名字在**编译期**改名：

```
-include tools/harness/guarded_heap.h \
-Dmalloc=wt_guard_malloc -Dfree=wt_guard_free \
-Drealloc=wt_guard_realloc -Dcalloc=wt_guard_calloc
```

harness 本体（`guarded_heap.c`）**不带这些宏**编译，否则它自己也会被改名而够不到 CRT。每次分配独占一段页，`free` 后整段 `PAGE_NOACCESS` 且**地址不回收**——这一点 是必需的：朴素 `VirtualFree` 会把同一地址在下次分配时还回来，UAF 于是对着"合法但 是垃圾"的内存跑完（CVE-2026-87933 在加隔离前就是这样观测到的）。

平台分工：**Windows 走预处理器改名**（`tools/harness/guarded_heap.{c,h}`），**Linux 走 `LD_PRELOAD` 拦截**（`tools/harness/linux/guarded_heap.c`，`dlsym(RTLD_NEXT, ...)` 拿真实符号，`mmap` 单段 + `mprotect(PROT_NONE)` 隔离），后者用于只能跑预编译 `.so` 的场景。守卫堆一旦启用，`verify()` 直接把 `memory_checks` 置真——隔离本身就是一个内存检查器。

真库路径（预编译 OHOS ELF，宿主机跑不了）由 `tools/harness/ohos/run_real.sh` 承担：musl 加载器 + gdb 两轮 + `llvm-addr2line -i`，输出 JSON（`{"built","exit","crashed","offset","rounds","frames"}`）。凭证仍只经 `OSLOVELACE_API_KEY` 环境变量注入，不入库不入日志。

端到端（起草→四道闸门）实测约 30 s/案，对比手工复现约 2 小时。

3.16 Joern CPG 端到端实测与双引擎交叉（已实测）

两引擎各自出结论再交叉，避免单一引擎背书。**函数粒度并集实测**（2026-10-18，nstackx 子组件，`COMPLETENESS_PUSH_20261017.md`）：Joern 1,271 发现（J6×1224 + J8×47）对规则层 2,404；交集 752 函数（Joern 覆盖函数的 88% 双引擎互证）、Joern 独有 105、规则层独有 589；站点级零重合为锚点差异伪影（规则层报函数起始行、Joern 报解引用行），已在文档注明。 工作台 `--joern` 第 11 层首次完整落地的实测记录：`available=true`、`findings=1`（`J6_null_deref`，dsoftbus 靶区）。P6 规则层与 J6 Joern 层 形成双引擎交叉：规则层的语义分层与 CPG 层的结构路径互为印证，异常时 安全降级（规则层独立可用）。

3.17 自足构建链与真库 malformed 验证管线（已实现）

`tools/oh_build`（PR #133，已评审）固化"官方完整树 → 真库 malformed 验证" 四步管线：GN root-target 限定子图 → sweep_v3 清图（dupbuild/ 自盖章边 / 跨工具链碰撞） → 靶向 ninja → `run_malformed.sh`/`run_bt_dereg.sh` 验证（OH clang + musl sysroot + 加载器舞蹈 + 证据哈希封存）。已知坑清

单 （git-lfs、ninja 探针盲区、asmjit/zydis 上游缺口、Rust std 暂存）随工具包 入库。实证产出：udmf（`run_malformed.sh`，exit 139）与 bluetooth 服务端 DeregisterServerObserver（`run_bt_dereg.sh`，exit 139 + 符号化归因 `bluetooth_socket_server.cpp:54`，PR #135）。

3.18 分支钉版实证与影响矩阵校准（已实现）

`kb/third_party_targets/branch_pins.json`：五个发布分支 + master 的 manifest（ohos/ohos.xml）钉住 ref 在 **gitcode（manifest 声明的 remote）** 解析，逐一浅取 README.OpenSource 的 Version Number——实证修正了旧手工表 的多处估值（v4.1 ffmpeg=n4.1→4.4.1、v6.0=n6.1.1→5.1.4、curl v6.0= 8.11.0→8.8.0 等）。结合 kb 受影响区间，16 案（cJSON / curl×2 / ffmpeg×12 / openssl）六分支全部落定（🔴 或 not_affected 附证据链），待定格清零； iamf 依特性年表（n6.1 引入）判低版本分支不含缺陷代码。影响口径仍由 契约测试锁定"三出口同输入同判定"。**证据分层**（2026-10-18 交叉复核 F08 响应）： 🔴 中钉版联接/符号探测类格子显式标注弱于检出/复现级实证（矩阵图例脚注）， 分支引用名→SHA 固化列为后续项。

3.19 P8 分支/闭包敏感化（已实现，dsoftbus 448→69）

同函数 UAF 启发式的两类系统性误报形态抑制（作品报告"不足#2"闭环）：

- **分支互斥 free**：逃逸终止（free 所在分支以独立无条件 `return` 终止—— 交叉复核 F04a 后仅认同行无条件关键字且括号平衡的独立语句，花括号/无花括号 两形态一致）与 else 兄弟臂使用；
- **回调闭包**：使用位于 lambda 体内而 free 在闭包外（**立即调用闭包 `}()` 不抑制**——同次执行，复审 F04b）；
- **置空时序**（F04c）：free 后的 `p = NULL` 仅作用于其之后的使用， 置空自身的匹配不计使用；被置空排除的使用进抑制日志（reset-before-use）。

零过抑制设计：goto/break/continue 可跳转到后续使用，永不作为逃逸证据。 dsoftbus 全树实测：旧语义 448 → 新 69（移除 379 全为裁定形态；2,383 个使用点 带因抑制）。全部反例与保留形态由 `tests/test_external_review_fixes.py` 锁定。

3.20 种子化召回基准 v2 与召回守恒账本（已实现）

种子基准（`tools/seed_benchmark.py`，schema 0.2）：11 个正向种子 （GT1/GT2/GT3 裸解引用、P6 双形态、P7/P8 双形态、P4/P5/P1-CJ，OH 真实习语） + 5 个负向种子（守卫/逃逸分支/free-置空等）。v2 归属规则（交叉复核 F01 响应）： **每种子独立文件**，正匹配=模式族+文件+行窗，负种子声明 `silent_for` 族 （violation=真误报；by_design=P6 候选生成语义另行计数）；反向测试保证负文件 命中绝不抬升召回。**结果：召回 11/11、violations=0**（misattributed=3 为跨族 共检信息项）。基准驱动的真实召回修复：S03 暴露 P6 不认裸 `*p` 解引用——已修 （声明/乘法经前置 token 过滤排除；`* p`/`*(p)` 等价形态与标识符边界由复审 F05 补齐）。

召回守恒账本（`tools/funnel_ledger.py`）：分目录漏斗表 + P6/P8 抑制原因 分布。口径声明（非守恒断言）：lead 计数按函数合并、抑制计数按访问点累计， 两层单位不同，不做机器守恒校验（conservation_verified=false）； 抑制样本保留 供人工抽裁。全树账本数据采集中（`COMPLETENESS_PUSH_20261017.md`）。

4. 工程实现

4.1 规模与质量

- 测试：**1,248 项用例**（`unittest` 发现计数），CI（Ubuntu）全绿；v1.0 基线 1016 → v2.0 1,162 → v2.1 1,191 → v2.2 1,248 项（P6 契约 51 项、影响契约、检测契约、 P8 敏感化配对基准、种子基准 v2、交叉复

核反例回归、PoC 闸门 72 项）。

- **2026-09-29 全面代码审计与修复**：四子代理并行审查 + 逐项实证复现，发现并 修复 2 项 P0（影响矩阵跨组件基线兜底伪造全红、"11 层"死代码）、多项 P1/P2 （P9 格式参数位、P4 打分变量、P6 静默丢弃、cross_uaf 重复计数与别名错位、 llm_poc 注入/路径穿越、报告边界崩溃等），修复即配回归测试钉死。
- **2026-10 证据质量整改**：五份交付件口径统一；20 案 PoC/补丁全覆盖 （rscc/iamf 运行级差分与分配定点报告、curl/nvdec 环境门槛如实申报）； 证据质量矩阵实质率 54%→68%（○ 格清零）；分支钉版实证 + 原生四案符号探测后 影响**矩阵 20 案 × 6 分支全部判定**（含 udmf v5.1 负向发现）。
- **2026-10-18 交叉复核响应**：F01-F08 八项发现全部对照真实代码 复现成立并修复（基准归属 v2、P6 守卫确认收窄为独立无条件 return、P8 三处 抑制收窄、llm_verify 状态拆分、矩阵证据分层脚注）；20 项反例 +保留形态回归 锁定（`EXTERNAL_REVIEW_RESPONSE_20261018.md`）。AST/tokenizer 级访问识别与 账本机器守恒校验如实列为赛后项。
- 分支治理：单一交付分支（master），PR 独立审查、CI 全绿后合并，合并后自动 删除头分支；行尾策略（.gitattributes）与本地产物忽略（.gitignore）就位。

4.2 数据契约族（schema 清单）

`audit-result-0.3` 、 `evidence-bundle-1` 、 `workflow-checkpoint-1` 、 `engine-profile-0.1` 、 `llm-review-plan/draft-0.1`、`review-evidence-handoff-0.1`、 `knowledge-base-0.1`、`kb-manifest-0.1`、`partner-evidence-sidecar-0.1` 、 `cjson-observed-matrix-0.1` 、 `case-specific-validation-0.2` 、 `kb-version-probe-0.1` 、 `wildtrace-competition-report-0.1` 等——全部带版本号与 向后兼容策略（旧格式显式拒绝）。

4.3 CLI 入口速查（节选）

```
# 自动化管线（默认 10 层；--joern 启用第 11 层，需 WSL /opt/joern）
python -m ohvul.autopilot <root> --output <dir> [--min-confidence N] [--joern]
# LLM PoC 生成（选配，需 OSLOVELACE_API_KEY）
python -m ohvul.llm_poc findings.json <source_root> --output llm_pocs
# 影响矩阵与交付件
python -m ohvul.impact_matrix --cases cases --output docs/deliverables/IMPACT_MATRIX
python -m ohvul.vuln_report --cases cases --output docs/deliverables/VULN_REPORT.md --force
python -m ohvul.competition_report --cases cases --output out/report.json --markdown out/report.md
# 工具链预检（演示第一步）
python -m ohvul.toolchain --output preflight.json
# 离线管线与知识库
python -m ohvul.cli <root> --output ... --checkpoint ... --vuln-db ...
python -m ohvul.kb_sync build/verify ...
python -m ohvul.llm_review --plan|--run --allow-network|--verify ...
python -m ohvul.benchmark <cases.json> [--scanner ...]
```

---

5. 实测证据汇总（截至 2026-10-22，全部可复验）

项目	数字
回归测试	**1,248** 项用例（`unittest` 发现计数）；CI Ubuntu 全绿
真实 OH 文件分析	44 文件（3 组件，profile 来源校验通过）
运行级证据案例	cJSON 矩阵 26/26；独立复现 26/26（双重复现）

项目	数字
CVE 知识库	375,265（全量，本地）+ 72（精选）+ 446（靶区，兼矩阵数据源）
Autopilot 实测（现役口径）	dsoftbus 681,529 行 / 651.7s / 3,033 lead / 峰值 176.4MB
（`PERFORMANCE.md` 2026-09-30 复测；含 cross_uaf 去重与判定收紧）	
CVE 富集组件批量扫描（2026-10-19 夜运行）	6 组件 2,339,964 行 / 8,750.8s / 67,453 lead（高危 417）/ P6 抑制 27,204
累计覆盖	dsoftbus 68 万 + FFI 63 仓 17 万 + CVE 富集 234 万 = **3,195,394 行**
P6 语义分层漏斗	2,662 条治理后候选中 P6 族 2,224 条 → **13**（Null 确定性 > 语义类 > 已知可空 > 参数 > 基线）
GT1/GT2/GT3 修复对回放	三对全部命中→消失零误伤（§3.14 表）
Joern CPG 实测	管线内端到端：available=true / findings=1（J6_null_deref）；
独立记录：CPG 构建 158s + 查询 58s	
LLM PoC 四道闸门	端到端 ~30s/案；cJSON 默认堆 `return 0`、隔离堆 SIGSEGV → `confirmed`；
UDMF 首版 139 但帧落 `SetDataParameters` → `wrong_site`，改 `0x1718a` 后 `confirmed`	
PoC 闸门回归	`test_llm_poc` 28 + `test_poc_verify` 24 + `test_poc_real` 20 = **72** 项
真库 malformed 验证	udmf：run_malformed.sh exit 139；bluetooth 服务端：
run_bt_dereg.sh exit 139 + 符号化 `bluetooth_socket_server.cpp:54`（PR #135）	
影响矩阵	**20 案 × 6 分支全部判定**（第三方 16 案钉版联接 + 原生四案符号
探测，含 udmf v5.1 负向发现）；证据分层见矩阵图例（钉版联接/符号探测弱于	
检出/复现级，2026-10-18 脚注）	
证据质量矩阵（2026-10-18）	20 案 × 12 字段：●149 / ◐70 / ○0，实质率 **68%**（PoC 与
补丁字段全覆盖；五案新增运行级证据）	
漏洞案例与工单	20 案（16 个第三方 CVE 缺口 + 4 项原生组件缺陷：ipc / hiappevent / kv / udmf）；
平台工单 17 张提交待受理；第 18 张（bluetooth 服务端）材料齐备	

项目	数字
LLM 全量审查（2026-10-17 执行）	26/26 组有效草稿（glm-5.3）；46 调用 / 66,782
tokens；终轮 verify completed；全部 final_verdict=undetermined	
P8 敏感化（2026-10-17/18）	dsoftbus 旧语义 448 → 新 69（379 移除全为裁定形态；
复审 F04 收窄后反例回归锁定）	
种子化召回基准 v2	召回 11/11、violations=0（独立文件归属；负种子 silent_for）
双引擎并集（nstackx，2026-10-18）	函数粒度交集 752（Joern 覆盖 88% 互证）、
Joern 独有 105、规则层独有 589（锚点差异已注明）	
交叉复核响应（2026-10-18）	F01-F08 八项全复现全修复；20 项反例回归；
全量 1,191 测试绿（10-18 口径）	
全树扫描	进行中（34.4M 行产品代码面，双进程后台；完成后补
`FULLTREE_FUNNEL_LEDGER` 与覆盖率口径——本表不预支数字）	

6. 创新点（八项，与作品报告 §4 对齐）

1. **多模式族通用检测管线**：13 个模式族（通用 10 + FFI 3）+ Joern CPG 双引擎； 仓颉 FFI 边界新攻击面定义与首发挖掘为特色方向之一。
2. **ABI 契约感知检测与极性宏守卫**：规则从 FFI ABI 契约推导而非通用模式； 68 万行实测精确消除 229 条误报，每个判定可解释、每次修正可验证。
3. **指针状态模型与可证明性排序**：Null/NonNull/MaybeNull 时间线 + 语义分层 漏斗（dsoftbus 2,662→13）+ 每个不发射访问的带因抑制日志——误报治理从 "调阈值"变为"可审计的语义论证"（交叉复核后守卫确认进一步收窄为独立 无条件 return，伪安全反例回归锁定）。
4. **双引擎交叉**：P6 规则层语义 × Joern CPG 结构互证（函数粒度交集 752、 Joern 独有 105，锚点差异如实注明）。
5. **真库验证阶梯**：官方完整树零源码改动构建 + dlopen/musl + exit 139 + gdb 帧归因（四项真库 SIGSEGV）；两项确定性判读技术（分配器填充差分、 ASan 分配上限定点拦截）；round-trip 自毁论据；自足构建链工具化。
6. **LLM 的两个受控方向**：PoC 四道闸门（~30s/案，检测器教 LLM；自模仿守卫 堵掉"抄函数体"的套套逻辑，`wrong_site` 档区分"崩在库里"与"崩在这一行"）+ "供信号不裁决"治理（26/26 草稿全 undetermined、密钥运行时、注入防御、 输出屏蔽）——单模型全量已执行，双模型因密钥不可行如实降级。
7. **已知答案回放的检测质量度量 + 种子化召回基准**：严格修复对（GT1/GT2/GT3 命中→消失零误伤）+ 种子基准 v2（独立文件归属，召回 11/11、violations=0）； P8 分支/闭包敏感化同属此脉（448→69，反例回归锁定）。
8. **证据工程**：内容寻址证据链 + 离线复算 + 第二人独立复现 + 三出口一致性 契约 + 证据质量矩阵（实质率 54%→68% 可回归）+ 分支钉版实证（矩阵 20 案 全分支判定，证据分层如实标注）+ 诚实证伪入册。

（方法论母题：审计驱动的自我纠错——2026-09-29 全面审计、2026-10-18 交叉 复核 F01-F08 全项响应，均以"复现→修复→反例回归"闭环，见 §4.1。）

7. 已知局限与未完成项（诚实清单）

- 漏洞确认口径：平台受理结果未回（17 张工单待受理中，催办已发）；本地证据 为真库级 SIGSEGV（ipc/hiappevent/udmf/bt 四项，帧归因在库内）与公开案例 库级复现，产品整机环境复现未建立。
- DEMO 视频待录制（VIDEO_SCRIPT v2 已按当前事实修订完毕，可直接录制）。
- 双模型交叉验证不可行（仅存 GLM 密钥），单模型全量已执行；比对器保留备用。
- cfhd 案运行级观测需上游系数样本（驱动与口径已备）；nvdec 需 GPU/NVDEC 宿主、curl 案需 Schannel/GnuTLS 后端（如实申报，不冒充复现）。
- 访问识别为词法层（含边界/空白/括号等价形态），AST/tokenizer 级识别为 赛后项（交叉复核 F05 长期建议，已采纳方向）；`(a) *p` 类 `)` 前导形态 保留（deref-assign 合法形态，指针类型下乘法读法不可编译）。
- 账本未做机器守恒校验（两层计数单位不同——lead 按函数合并、抑制按访问点 累计）；口径已声明，统一单位重铸为赛后项（复审 F06 建议）。
- 影响矩阵 🔴 中钉版联接/符号探测类格子弱于检出/复现级实证（图例脚注分层）； 分支引用名→SHA 固化列为后续项（复审 F08）。
- 全树扫描（34.4M 行产品代码面）进行中——完成前不主张覆盖率数字。
- 内置正则扫描仅作初筛；跨函数分析为轻量别名级，非完整指针分析； LLM PoC 执行（llm_verify 的 WSL subprocess 编译/执行）无网络/文件系统 隔离配置，接入自动化前需显式授权与执行隔离（复审 F07 提示，现行使用 均为人工触发）。

8. 附录：关键文档索引

- 交付件：`docs/deliverables/`（TECHNICAL_SPEC、VULN_REPORT、IMPACT_MATRIX、PERFORMANCE、VIDEO_SCRIPT、DEMO_RUNBOOK）；
- 状态台账：`docs/status/`（CASE_LEDGER、SUBMISSION_QUEUE、PROJECT_STATE 等）；
- 审计记录：`docs/analysis/`（含 2026-09-29 L4 三级审计、REAL_BUILD_PROBE、 cJSON 运行案例、独立复现、各轮代码审查）；
- 设计稿：TRIAGE_SIGNALS_DESIGN.md、EXECUTION_SANDBOX_DESIGN.md；
- 工程契约： docs/engineering/*（LLM_REVIEW、REVIEW_HANDOFF、KB_VERSION_PROBE、EVIDENCE_VALIDATION 等）；
- 变更史：CHANGELOG.md（追加一至二十，含全部审计修复记录）；
- 2026-10 增补：`EXTERNAL_REVIEW_RESPONSE_20261018.md`（交叉复核响应）、`COMPLETENESS_PUSH_20261017.md`（完整性推进）、`LLM_REVIEW_RUN_20261017.md`（LLM 全量执行）、`SECURITY_IMPACT_QUALIFICATION_20261017.md`（DoS 定性）。